

# S2S-94 LAB: Retiring AWS for Azure

## — Moving the Environment and the Workloads

---

**Series:** S2S — SaaS to SaaS Migration | **Reference:** 94 — AWS-to-Azure Cloud Retirement LAB | **Created:** September 2026 | **Last Updated:** 09/28/2026

## Overview

---

An appendix LAB for an organization that is **completely retiring AWS in favor of Azure**. Two moves happen at once, and they are easy to conflate:

1. **The Dynatrace environment moves** from an AWS-hosted SaaS cluster to an Azure-hosted one. That is a tenant move: a new target environment, and everything in S2S-01 to S2S-09 applies.
2. **The monitored workloads move** from AWS to Azure. That is a monitoring change: new cloud connections, new log ingest paths, agents on new hosts.

Because workloads move in waves, there is a **dual-cloud window**: the new Azure-hosted environment monitors workloads still running on AWS *and* workloads already on Azure. The window ends when the last AWS workload is gone, and only then are the AWS connection and the AWS-hosted source environment retired.

This LAB is the ordered runbook for that case. It does not repeat the nine steps; it sequences them for this scenario and adds what is specific to it. Two FAQ entries frame it: **FAQ-25** (what carries over to a new tenant) and **FAQ-17** (the cutover invariants — Go/No-Go gate, parallel-run window, rollback triggers, decommission).

---

## Table of Contents

---

1. [The Scenario in One Table](#)

2. [Phase 0 — Decide and Provision the Azure-Hosted Target](#)
3. [Phase 1 — Identity, Access, and Configuration](#)
4. [Phase 2 — Connect Both Clouds to the Target](#)
5. [Phase 3 — Agent Waves](#)
6. [Phase 4 — Log and Event Ingest Cut-over](#)
7. [Phase 5 — Validate the Dual-Cloud Window](#)
8. [Phase 6 — Decommission AWS and the Source Environment](#)
9. [Open Questions and Unverified Items](#)
10. [Summary](#)

---

## Prerequisites

---

| Requirement                 | Details                                                                                                 |
|-----------------------------|---------------------------------------------------------------------------------------------------------|
| <b>S2S series</b>           | S2S-01 to S2S-05 read; S2S-06 §1–2 for cloud connections                                                |
| <b>Source environment</b>   | The AWS-hosted Dynatrace SaaS environment, admin access                                                 |
| <b>Commercial agreement</b> | An Azure-hosted target agreed with your Dynatrace account team (Azure regions are available on request) |
| <b>AWS access</b>           | Rights to deploy CloudFormation stacks in every monitored AWS account                                   |
| <b>Azure access</b>         | Rights to create a service principal and assign it on each monitored subscription                       |
| <b>Tooling</b>              | Monaco v2.x, Terraform (IAM), <code>oneagentctl</code> on hosts, <code>kubectl</code> for DynaKube      |

## 1. The Scenario in One Table

|                              | Before                                   | Dual-cloud window                                                                        | After                    |
|------------------------------|------------------------------------------|------------------------------------------------------------------------------------------|--------------------------|
| <b>Dynatrace environment</b> | AWS-hosted source                        | Azure-hosted target is primary; source readable, no longer ingesting from migrated waves | Azure-hosted target only |
| <b>AWS workloads</b>         | Monitored by the source                  | Re-pointed to the target, wave by wave, while they still run                             | Gone                     |
| <b>Azure workloads</b>       | — (or monitored by the source)           | Monitored by the target from day one                                                     | Monitored by the target  |
| <b>AWS connection</b>        | On the source                            | On the <b>target</b> (and removed from the source)                                       | Removed                  |
| <b>Azure connection</b>      | — (or classic integration on the source) | On the target, one subscription at a time                                                | On the target            |

**Sequencing recommendation.** Stand up the Azure-hosted target **before** the workload migration starts, so that new Azure workloads install OneAgent against the target directly. A workload that moves to Azure while still reporting to the AWS-hosted source has to be re-pointed a second time later. This follows from the one-destination rule for OneAgent (FAQ-25 §6) rather than from a Dynatrace document, so treat it as planning guidance.

## 2. Phase 0 — Decide and Provision the Azure-Hosted Target

Dynatrace documents that *"Data is stored in Amazon Web Services (AWS), Microsoft Azure, or Google Cloud data centers"*, and lists Azure regions (East US, West US 3, West Europe, Canada Central, UAE, Switzerland North, Australia East) as *"Available on request. Talk to your Dynatrace sales contact."* It also states *"Environments hosted on Azure use dedicated Azure storage accounts"* — useful for the data-residency conversation.

Choose between the two ways of getting an Azure-hosted environment (the full comparison is in **S2S-04** §1):

| Decision                              | Standard SaaS on an Azure region              | Azure Native Dynatrace Service                                                                                                        |
|---------------------------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Provisioned by                        | Your Dynatrace account team                   | You, from the Azure portal — <i>"available via a private offer"</i>                                                                   |
| Account                               | Settled with the account team                 | <i>"The integration will create a new Dynatrace environment and account; it can't run on an existing Dynatrace SaaS environment."</i> |
| Billing                               | Your Dynatrace agreement                      | <i>"your Dynatrace license consumption becomes a part of your regular Azure bill"</i>                                                 |
| SSO                                   | SAML — Microsoft Entra ID is a documented IdP | Entra ID; <i>"The integration works across a single Entra ID environment."</i>                                                        |
| Azure Cloud Platform Monitoring on it | Documented                                    | <b>Not stated</b> — see §9                                                                                                            |

### Runbook:

| #   | Action                                                                          | Done when                                                                            |
|-----|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 0.1 | Agree region and path with the account team and your Azure commercial owner     | Written decision, including how the source commitment and target consumption overlap |
| 0.2 | Provision the target                                                            | Environment reachable; admin can sign in                                             |
| 0.3 | Record both environments' platform state (Classic vs Latest surfaces, versions) | Recorded in the migration plan ( <b>S2S-01</b> §8)                                   |
| 0.4 | Freeze the source configuration once the export is taken                        | Freeze communicated ( <b>S2S-04</b> §6)                                              |

#### Sources:

- [Data security controls \(DT docs\)](#) — *"Environments hosted on Azure use dedicated Azure storage accounts."*
- [Azure Native Dynatrace Service \(DT docs\)](#) — *"When you purchase Dynatrace SaaS through the Azure Marketplace, your Dynatrace license consumption becomes a part of your regular Azure bill."*

### 3. Phase 1 — Identity, Access, and Configuration

| #   | Action                                                                                                                                                       | Reference                          | Done when                                                              |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|------------------------------------------------------------------------|
| 1.1 | Configure SAML SSO with Microsoft Entra ID as a <b>new</b> enterprise application for the target                                                             | <b>S2S-04</b> §2                   | Two users in different groups sign in with the expected permissions    |
| 1.2 | Keep group claims under Entra ID's limit — <i>"The number of groups emitted in a token is limited to 150 for SAML assertions"</i>                            | <b>S2S-99</b> #43                  | Group claims arrive for your largest user                              |
| 1.3 | Rebuild the IP allowlist on the target (per environment, inbound)                                                                                            | <b>S2S-04</b> §2                   | CI/CD runners and admins reach the target API                          |
| 1.4 | Deploy IAM with Terraform                                                                                                                                    | <b>S2S-04</b> §2, <b>S2S-05</b> §3 | Groups, policies, bindings applied                                     |
| 1.5 | Export with <code>monaco download</code> , deploy with <code>monaco deploy</code> — not the SaaS Upgrade Assistant, which is documented for a Managed source | <b>S2S-10</b>                      | Dry-run clean; deploy to the target succeeds                           |
| 1.6 | Deploy ActiveGates the target needs (routing, extensions, private synthetic locations) — on Azure where the workloads are going                              | <b>S2S-04</b> §4                   | <code>smart scapeNodes</code><br>"ACTIVEGATE" lists them in the target |

**Unverified — egress addresses.** If your firewalls, webhook receivers or ITSM tools admit Dynatrace by source IP, the Azure-hosted cluster's egress addresses will differ from the AWS-hosted source's. No primary source for them was found for this LAB; request them from Dynatrace before cutover rather than reusing the old list.

**Sources:** [Group claims \(Microsoft Learn\)](#) — *"The number of groups emitted in a token is limited to 150 for SAML assertions and 200 for JWT, including nested groups."*, [IP allowlist \(DT docs\)](#).

## 4. Phase 2 — Connect Both Clouds to the Target

The target needs **both** connections during the window: AWS for the workloads that have not moved yet, Azure for the ones that have. Both are Latest Dynatrace Cloud Platform Monitoring connections, and neither needs an ActiveGate for polling (**S2S-06** §1).

**Azure — switch per subscription, never overlap.** The docs: *"Do not onboard Azure subscriptions already monitored by the classic Azure integration, and avoid monitoring the same subscription across multiple Azure connections—both increase the risk of API throttling and service interruptions."* If the source already monitors some Azure subscriptions, move each one in a single change window:

| #   | Action                                                                                                              | Done when                                        |
|-----|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| 2.1 | Create a <b>dedicated</b> service principal for the target — <i>"Do not share it across Dynatrace environments"</i> | Principal exists, assigned on the subscription   |
| 2.2 | Remove the subscription from the source's integration (classic or Latest)                                           | Source no longer polls it                        |
| 2.3 | Add the subscription to the target's Azure connection                                                               | Its resources appear in the target (query in §7) |

**AWS — a new connection from the target.** *"All AWS connection creation methods are powered by CloudFormation"*: create the connection in the target and deploy its stack into each account. The AWS onboarding page read for this LAB carries no double-monitoring warning; keep any period in which both source and target poll the same account short, and confirm cost and API-quota implications with your account team.

| #   | Action                                                                               | Done when                          |
|-----|--------------------------------------------------------------------------------------|------------------------------------|
| 2.4 | Create the AWS connection in the target; deploy its CloudFormation stack per account | AWS resources appear in the target |
| 2.5 | Remove the AWS integration from the source once the target shows the same accounts   | Source stops polling AWS           |

### Sources:

~ [Create your first Azure connection \(DT docs\)](#) — *"Use a dedicated service principal exclusively for Dynatrace. Do not share it across Dynatrace environments or use it for other non-Dynatrace workloads."*

– [AWS Cloud Platform Monitoring \(DT docs\)](#) — "All AWS connection creation methods are powered by CloudFormation as Infrastructure-as-Code (IaC) engine."

## 5. Phase 3 — Agent Waves

Two kinds of wave run side by side:

| Wave type                                                         | What happens                                                                                | Mechanism                                                                                                                                                      |
|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Re-point</b> — AWS workloads still running                     | The host stays on AWS; its OneAgent moves to the target                                     | <code>oneagentctl --set-server=... --set-tenant=... --set-tenant-token=... --restart-service</code> , then restart monitored applications ( <b>S2S-05 §5</b> ) |
| <b>Re-platform</b> — workloads rebuilt on Azure                   | New hosts on Azure; OneAgent installed from the target                                      | Standard OneAgent installation against the target — no re-point                                                                                                |
| <b>Kubernetes</b> — EKS clusters still running / new AKS clusters | EKS: delete the DynaKube and apply one for the target. AKS: apply a DynaKube for the target | <i>"starting with Dynatrace Operator version 1.3.0, editing spec.apiUrl is not allowed"</i> ( <b>S2S-05 §6</b> )                                               |

A host reports to one environment at a time, so the source/target overlap is **per wave** (**FAQ-25 §6**). After each wave, check where hosts are — by cloud and region — in the target:

```
// Target: hosts by cloud provider and region – run after every wave
// cloud.provider is aws / azure / gcp on cloud-hosted hosts; the region field
// differs per cloud
// (aws.region vs azure.location), so coalesce them into one column.
smartscapeNodes "HOST", from:-2h
| fieldsAdd provider = coalesce(cloud.provider, "none (on-premises or
undetected)"),
    region = coalesce(aws.region, azure.location)
| summarize hosts = count(), by:{provider, region}
| sort provider asc, hosts desc
```

Over the window, the `aws` rows should shrink and the `azure` rows grow; the sum across the source and the target should match your discovery baseline (allowing for hosts that

were retired rather than migrated).

## 6. Phase 4 — Log and Event Ingest Cut-over

| Source of data                                    | On the source                                    | On the target                                     | Cut-over rule                                                                                                                             |
|---------------------------------------------------|--------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| OneAgent log collection                           | Follows the agent                                | Follows the agent                                 | Moves with each wave automatically                                                                                                        |
| AWS CloudWatch logs                               | Source's forwarder or Firehose streams           | Firehose streams from the target's AWS connection | Switch per log group with its workload's wave; retire at AWS decommission                                                                 |
| Azure resource / activity logs                    | Classic log forwarder, if any                    | Event Hubs ingest on the target                   | Switch per subscription with 2.2–2.3                                                                                                      |
| Azure resource events                             | —                                                | Event Grid system topics                          | Enable per subscription                                                                                                                   |
| Classic Azure log forwarder (if kept as fallback) | Direct via Cluster API, or through an ActiveGate | Redeployed against the target                     | <i>"Logs older than 24 hours are rejected"</i> — switch within the change window; a forwarder reointed after a longer gap cannot backfill |

OpenPipeline rules and bucket routing that match on AWS log sources need Azure equivalents before the Azure ingest starts, or those records land in the default buckets (S2S-07).

### Sources:

- [AWS Cloud Platform Monitoring \(DT docs\)](#) — *"Subscribe CloudWatch log groups to auto-generated Firehose streams for immediate ingestion and analysis."*
- [Azure Cloud Platform Monitoring \(DT docs\)](#) — *"Subscribe Azure resources to Event Grid system topics and forward resource lifecycle events such as blob creation and deletion, resource group changes, and service health alerts to Dynatrace."*
- [Set up the Azure log forwarder \(DT docs, Dynatrace Classic\)](#) — *"Logs older than 24 hours are rejected (considered too old by the Dynatrace log ingest endpoint)."*

## 7. Phase 5 — Validate the Dual-Cloud Window

---

Run these in the **target** throughout the window. First, connection coverage — cloud resources per provider:

```
// Target: cloud resources per provider, from the AWS and Azure connections
smartscapeNodes "*", from:-2h
| filter startsWith(type, "AWS_") or startsWith(type, "AZURE_")
| fieldsAdd provider = if(startsWith(type, "AWS_"), then: "aws", else:
"azure")
| summarize {resources = count(), resource_types = countDistinct(type)}, by:
{provider}
```

A provider with no row means its connection is missing or not yet polling. Next, compare what the connections see with what OneAgent sees — virtual machines from each connection against OneAgent hosts on each cloud:

```
// Target: VM coverage – connection-discovered instances vs OneAgent-monitored
hosts
smartscapeNodes "AWS_EC2_INSTANCE", from:-2h
| summarize instances = count()
| fieldsAdd resource = "AWS EC2 instances (AWS connection)"
| append [smartscapeNodes "AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES", from:-2h
    | summarize instances = count()
    | fieldsAdd resource = "Azure virtual machines (Azure connection)"]
| append [smartscapeNodes "HOST", from:-2h
    | filter in(cloud.provider, {"aws", "azure"})
    | summarize instances = count(), by:{cloud.provider}
    | fieldsAdd resource = concat("OneAgent hosts on ", cloud.provider)
    | fieldsRemove cloud.provider]
```

The two numbers per cloud are not expected to match — a connection sees every instance in its scope, OneAgent only the hosts you installed it on. What matters is the trend: on Azure, OneAgent hosts should grow with each re-platform wave; on AWS, both should fall toward zero.

Finally, the **baseline clock** from **FAQ-25 §5**: how much history each host has in the target. Migrated and newly built hosts start at zero, and their detectors are only as good as that history:

```
// Target: days of history per host (the FAQ-25 baseline clock)
// interval:24h rather than 1d – calendar durations are not supported here.
timeseries cpu = avg(dt.host.cpu.usage), from:-30d, interval:24h, by:
{dt.entity.host}
| fieldsAdd days_in_window = arraySize(cpu)
| fieldsAdd days_with_data = arraySize(arrayRemoveNulls(cpu))
| fields dt.entity.host, days_in_window, days_with_data
| sort days_with_data asc
| limit 20
```

Route the noise from low-history hosts to a staging channel rather than disabling alerting (FAQ-25 §5). Log ingest per cloud is a quick cross-check that Phase 4 is complete:

```
// Target: log records per cloud provider (last hour)
// cloud.provider is populated only on records that carry cloud context; a
large null row is
// normal (OneAgent-collected logs from on-premises hosts, API ingest, and
similar).
fetch logs, from:-1h
| summarize records = count(), by:{cloud.provider}
| sort records desc
```

## 8. Phase 6 — Decommission AWS and the Source Environment

AWS goes last, and in this order. First confirm, in the target, that no AWS host is still reporting — widening the window so a host that stopped recently is still listed:

```
// Target: AWS hosts seen in the last 7 days, split by whether they still
report
// still_reporting = false means the host was seen in the window but not in
the last 2 hours.
smartscapeNodes "HOST", from:-7d
| filter cloud.provider == "aws"
| fieldsAdd still_reporting = lifetime[end] > now() - 2h
| summarize hosts = count(), by:{still_reporting, aws.region}
| sort hosts desc
```

Without `from:-7d`, only nodes seen in the default window are returned, and a host that went quiet a day ago would not appear at all. The decommission gate is **no**

`still_reporting = true` rows — and a zero here is only meaningful if the same query returned AWS hosts earlier in the window (never trust a zero on its own).

| #   | Action                                                                                                                                                                                                     | Done when                                                     |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| 6.1 | Last AWS workload retired or moved                                                                                                                                                                         | Query above shows no <code>still_reporting = true</code> rows |
| 6.2 | CloudWatch log groups unsubscribed; Firehose streams idle                                                                                                                                                  | No AWS rows in the log query (§7)                             |
| 6.3 | Remove the AWS connection from the target, and remove what its CloudFormation stacks created in each account (deleting a stack removes the resources it created — confirm against your AWS change process) | No AWS resources in the connection-coverage query             |
| 6.4 | Stop ingest into the source: revoke its ingest tokens, disable its remaining integrations; its data stays readable for its bucket retention                                                                | <b>S2S-09</b> §5                                              |
| 6.5 | Export what must outlive the source (audit logs, key problem reports)                                                                                                                                      | <b>S2S-09</b> §5 pre-decommission checklist                   |
| 6.6 | Request deprovisioning of the AWS-hosted source environment from the account team                                                                                                                          | Formal confirmation received                                  |
| 6.7 | Remove the source's SAML application from Entra ID, revoke its tokens and OAuth clients                                                                                                                    | <b>S2S-09</b> §5                                              |

**FAQ-17** covers the invariants that apply to this final step as to any cutover: the rollback triggers are decided before the decommission starts, and the decommission is not reversible.

## 9. Open Questions and Unverified Items

| Item                                                                                       | Status                                                                                                        | What to do                                                   |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <b>Azure Cloud Platform Monitoring / Clouds app on an Azure-Native-created environment</b> | The Azure Native Dynatrace Service page (labelled Dynatrace Classic) does not mention either; read 09/28/2026 | Confirm with Dynatrace before choosing the Azure Native path |

| Item                                                                                                            | Status                                                                                     | What to do                                                                         |
|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>Egress IP addresses of the Azure-hosted cluster</b>                                                          | No primary source found                                                                    | Request them from Dynatrace; update firewall and webhook allowlists before cutover |
| <b>Parallel AWS polling from two environments</b>                                                               | No documented limit or warning found on the AWS onboarding page (the Azure page does warn) | Keep it short; confirm cost and API-quota implications with the account team       |
| <b>Commercial overlap</b> (source commitment vs target consumption; Azure Marketplace billing for Azure Native) | Not a documentation question                                                               | Settle with the account team in Phase 0                                            |

## 10. Summary

| Phase    | Outcome                                                                                                           |
|----------|-------------------------------------------------------------------------------------------------------------------|
| <b>0</b> | Azure-hosted target agreed and provisioned; path (standard vs Azure Native) chosen with the open question settled |
| <b>1</b> | Entra ID SSO, IP allowlist, IAM and configuration on the target; egress addresses requested                       |
| <b>2</b> | Target holds both an AWS and an Azure connection; no Azure subscription on two connections                        |
| <b>3</b> | AWS hosts re-pointed and Azure hosts installed against the target, wave by wave                                   |
| <b>4</b> | Log and event ingest follows the waves; classic forwarders switched inside the 24-hour window                     |
| <b>5</b> | Dual-cloud coverage and per-host history measured in the target                                                   |
| <b>6</b> | No AWS host reporting; AWS connection removed; source environment deprovisioned                                   |

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*